



WELCOME

Data Modeling 101

How do I model a schema for my application?



Celia Halenke
Associate Solutions Architect
@MongoDB



Julian Wingenfeld
Associate Solutions Architect
@MongoDB

Agenda

Modeling Methodology

Embedding vs. Linking

Data Modeling Patterns

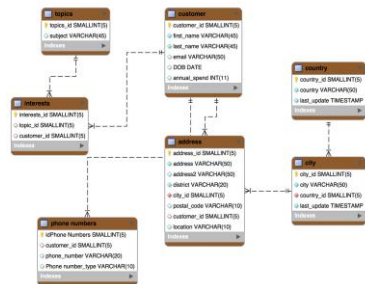
What's the value Setting up proper schema Design?



- Making your application **faster**
- **Saving Money**
- Make developers lives **easier**

**Data that is accessed together
gets stored together!**





Tabular (Relational) Data Model

Related data split across multiple records and tables

Contrasting data models

Document Data Model

Related data contained in a single, rich document

```
{
  "_id" : ObjectId("5ad88534e3632e1a35a58d00"),
  "name" : {
    "first" : "John",
    "last" : "Doe" },
  "address" : [
    { "location" : "work",
      "address" : {
        "street" : "16 Hatfields",
        "city" : "London",
        "postal_code" : "SE1 8DJ"}},
    + {...}
  ],
  "dob" : ISODate("1977-04-01T05:00:00Z"),
  "retirement_fund" : NumberDecimal("1292815.75")
}
```

Flexible: adapt to change



Add new fields dynamically at runtime

```
{
  "_id" : ObjectId("5ad88534e3632e1a35a58d00"),
  "name" : {
    "first" : "John",
    "last" : "Doe" },
  "address" : [
    { "location" : "work",
      "address" : {
        "street" : "16 Hatfields",
        "city" : "London",
        "postal_code" : "SE1 8DJ"},
      "geo" : { "type" : "Point", "coord" : [
        -0.109081, 51.5065752]}}],
    + {...}
  ],
  "dob" : ISODate("1977-04-01T05:00:00Z"),
  "retirement_fund" : NumberDecimal("1292815.75")
}
```

```
{
  "_id" : ObjectId("5ad88534e3632e1a35a58d00"),
  "name" : {
    "first" : "John",
    "last" : "Doe" },
  "address" : [
    { "location" : "work",
      "address" : {
        "street" : "16 Hatfields",
        "city" : "London",
        "postal_code" : "SE1 8DJ"},
      "geo" : { "type" : "Point", "coord" : [
        -0.109081, 51.5065752]}}],
    + {...}
  ],
  "phone" : [
    { "location" : "work",
      "number" : "+44-1234567890"},
    + {...}
  ],
  "dob" : ISODate("1977-04-01T05:00:00Z"),
  "retirement_fund" : NumberDecimal("1292815.75")
}
```

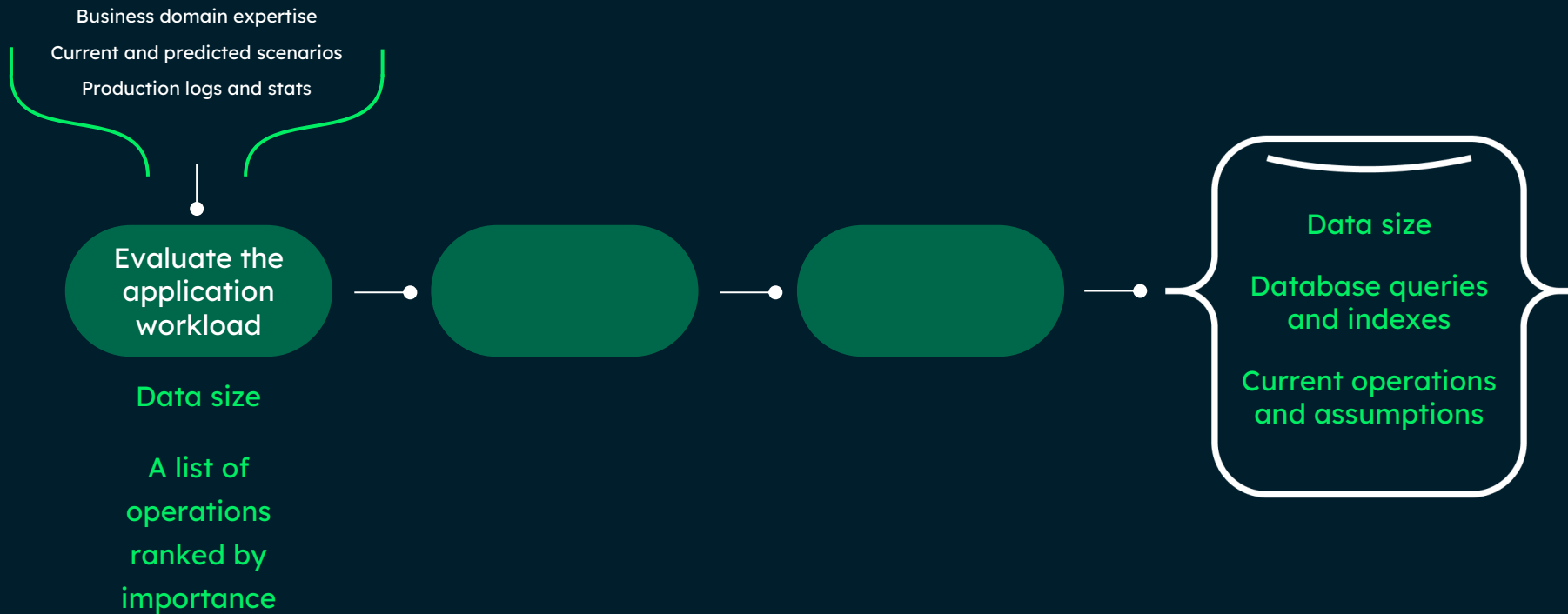


Modeling Methodology

Methodology to achieve a near magic almost formula

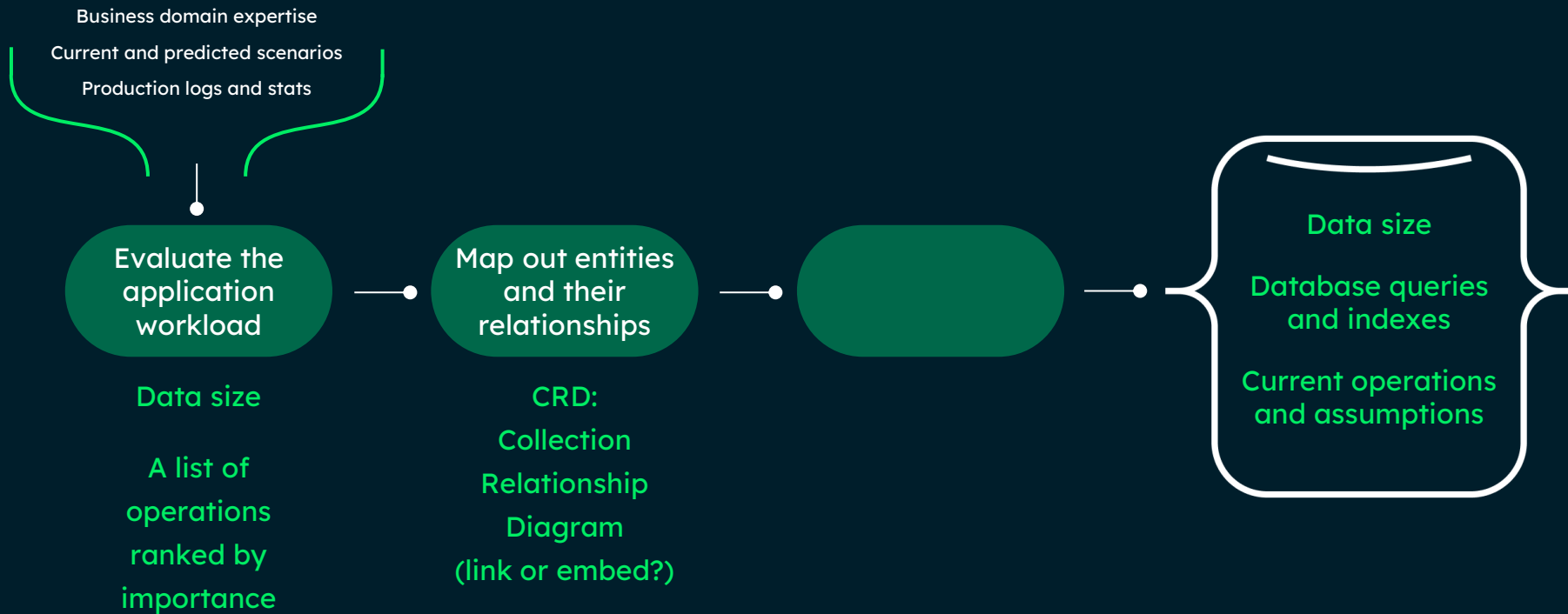


Step-by-step iteration



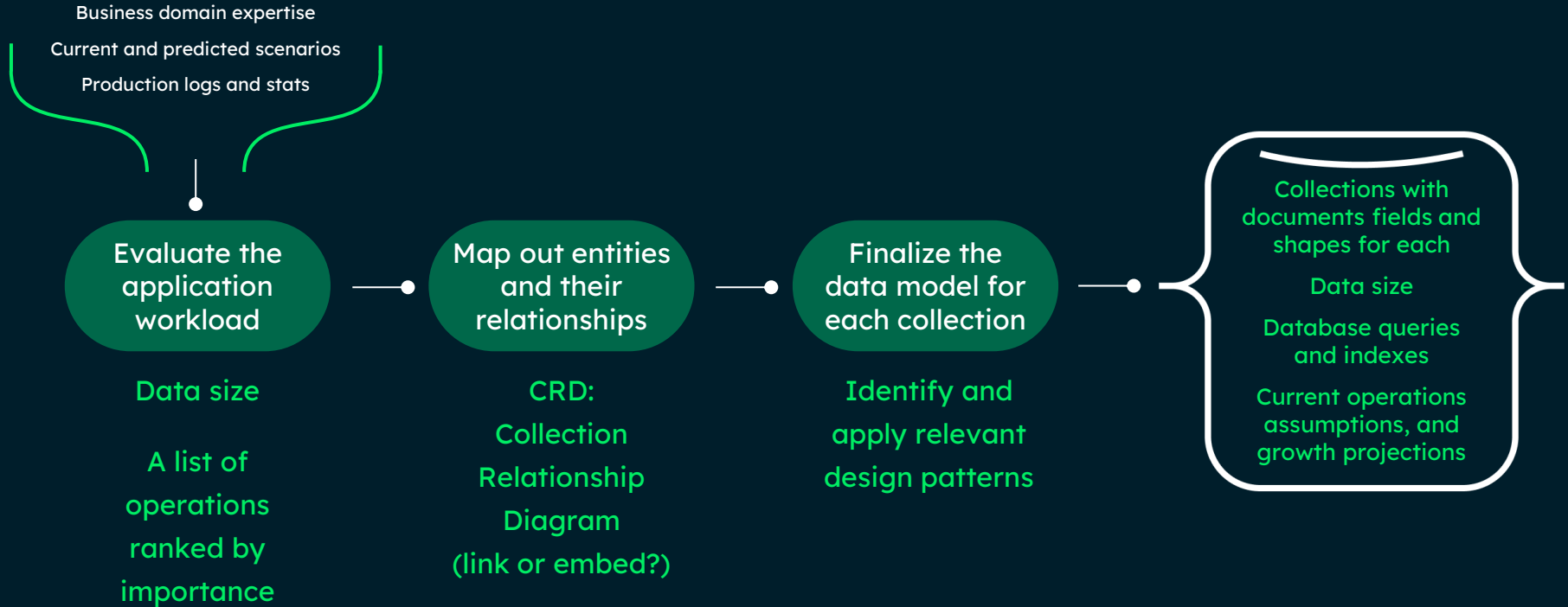


Step-by-step iteration





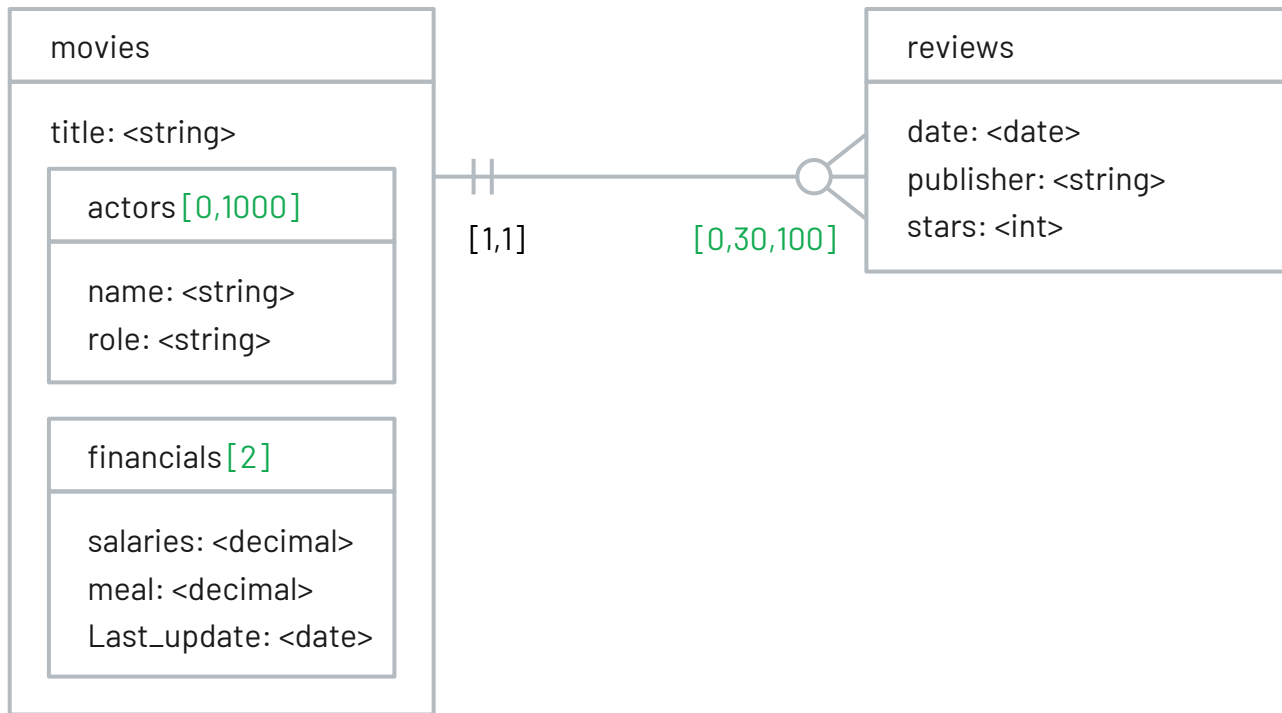
Step-by-step iteration



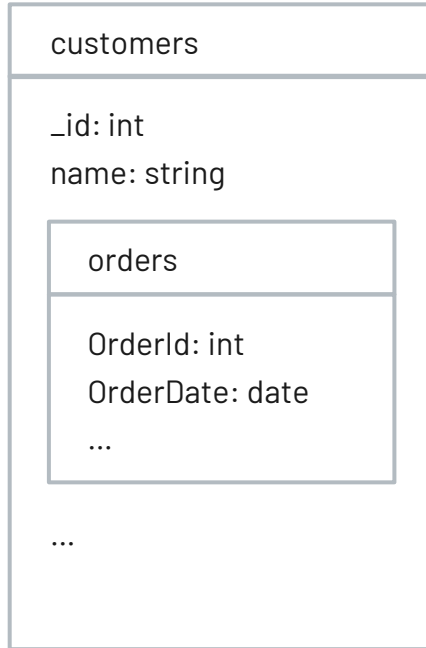


Embedding vs. Linking

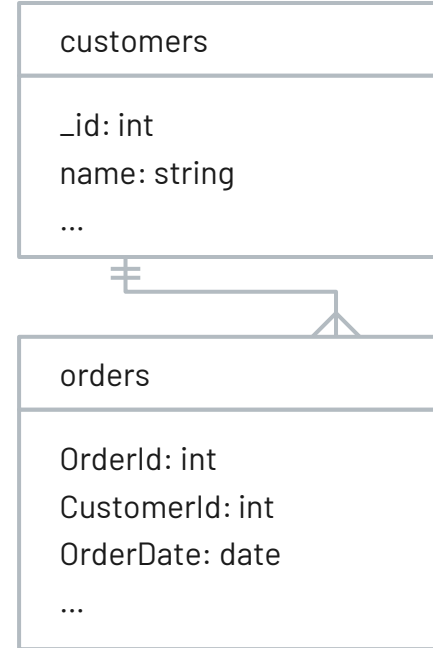
Embedding and linking example



Embedding



Referencing / linking





Embedding

```
{
  "first_name": "Paul",
  "surname": "Miller",
  "cell": "447557505611",
  "city": "London",
  "location": [45.123, 47.232],
  "profession": ["banking", "finance", "trader"],
  "cars": [
    {
      "model": "Bentley",
      "year": 1973
    },
    {
      "model": "Rolls Royce",
      "year": 1965
    }
  ]
}
```

Advantages

- You can retrieve all relevant information in a single query.
- Avoid implementing joins in application code or using **\$lookup**.
- Update related information as a single atomic operation. By default, all CRUD operations on a single document are ACID compliant.
 - You only have to change the information in one place
- However, if you need a transaction across multiple operations, you can use the **transaction operator.***



Embedding

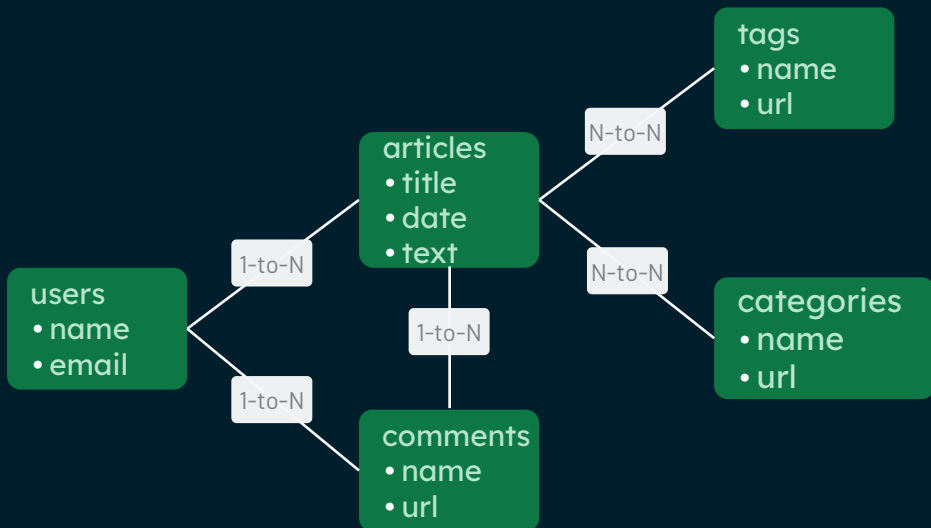
```
{
  "first_name": "Paul",
  "surname": "Miller",
  "cell": "447557505611",
  "city": "London",
  "location": [45.123, 47.232],
  "profession": ["banking", "finance", "trader"],
  "cars": [
    {
      "model": "Bentley",
      "year": 1973
    },
    {
      "model": "Rolls Royce",
      "year": 1965
    }
  ]
}
```

Limitations

- Large documents mean more overhead if most fields are not relevant.
 - You can increase query performance by limiting the size of the documents that you are sending over the wire for each query.
 - You can also enable over the wire compression.
- There is a 16-MB document size limit in MongoDB.



Linking

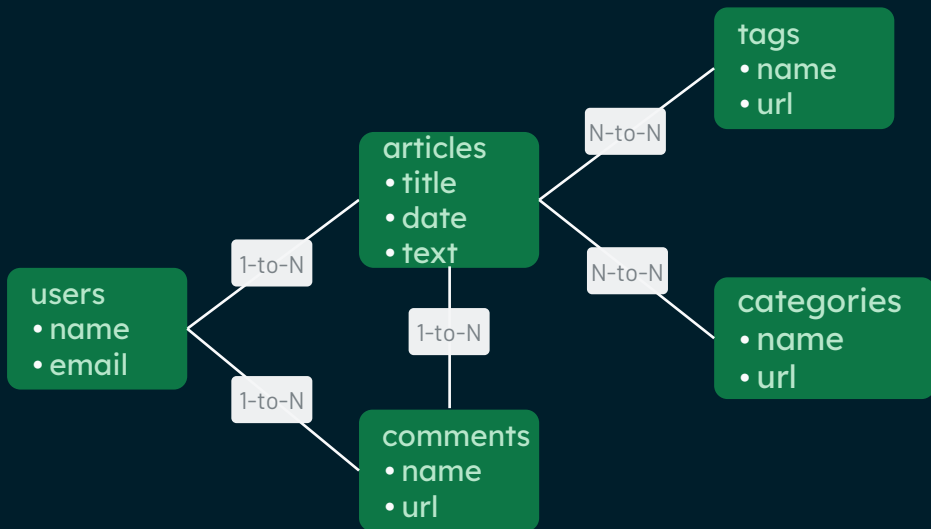


Advantages

- Smaller documents{typically}.
- Less likely to reach 16-MB-per-document limit.
- Infrequently accessed information not needed on every query.
- Reduce duplication of data.
 - It's important to note that data duplication should not be avoided if it results in a better schema for you application
 - Storage doesn't cost \$100 Billion dollars anymore.



Linking



Limitations

- In order to retrieve all the data in the referenced documents, a minimum of two queries or \$lookup is required to retrieve all the information.
- Changing one piece of data will usually result in many documents being updated.

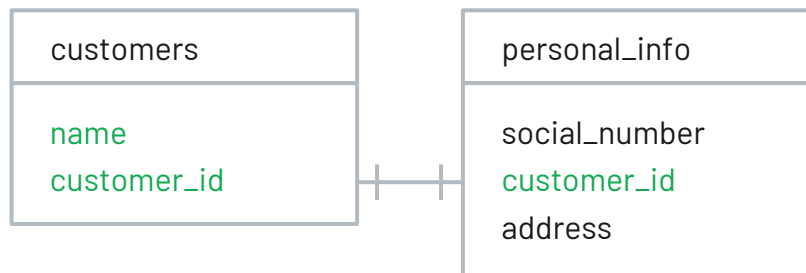
Relationships

One-to-One (1-1)

One-to-Many (1-N)

Many-to-Many (N-N)





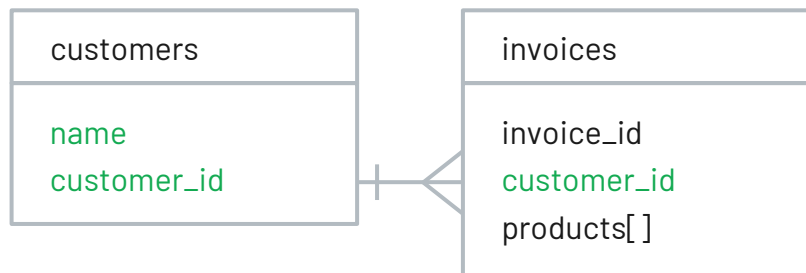
Relationships

One-to-One (1-1)

One-to-Many (1-N)

Many-to-Many (N-N)





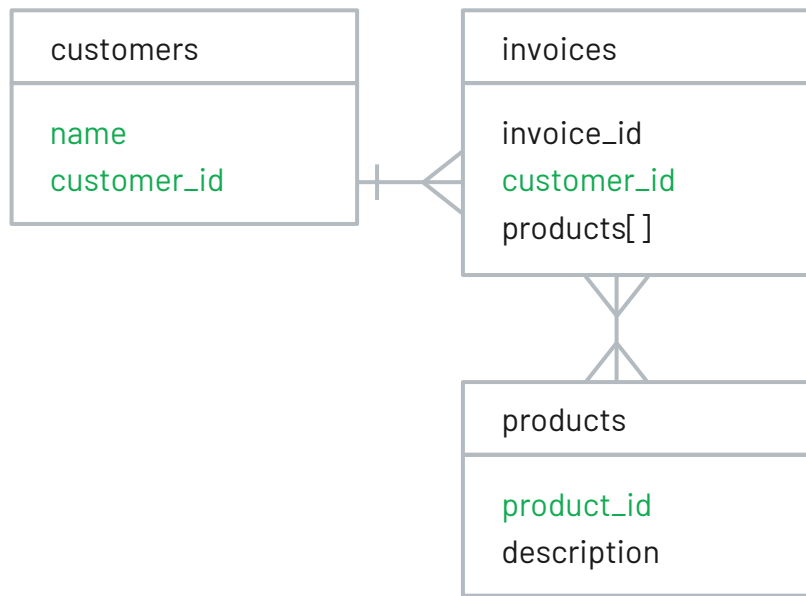
Relationships

One-to-One (1-1)

One-to-Many (1-N)

Many-to-Many (N-N)





Relationships

One-to-One (1-1)

One-to-Many (1-N)

Many-to-Many (N-N)



Schema Design - Link or Embed ?

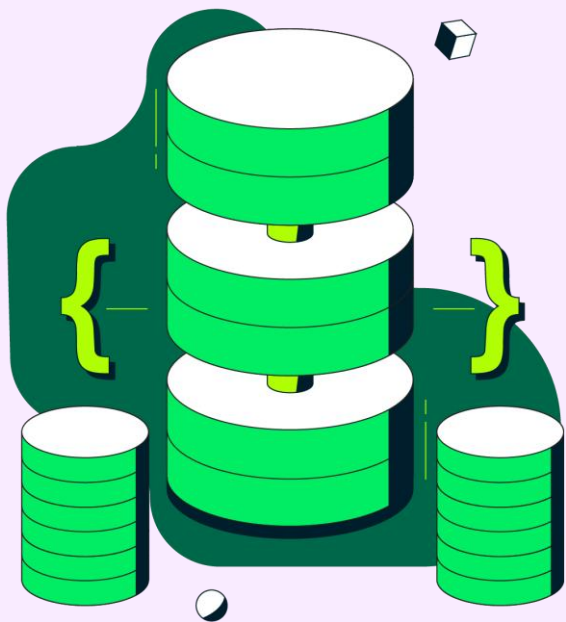
Recap:

- **One-to-One** - Prefer key value pairs within the document
- **One-to-Few** - Prefer embedding
- **One-to-Many** - Prefer embedding
- **One-to-Thousands/millions** - Prefer Referencing
- **Many-to-Many** - Prefer Referencing





Recommendation 1



Favor embedding unless there is a compelling reason not to.



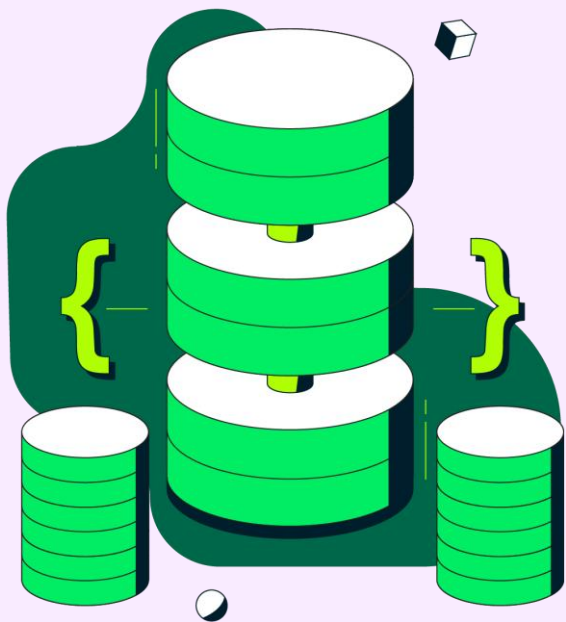
Recommendation 2



Avoid \$lookups if possible, however, use them if they can provide a better schema design. Two queries is also a very acceptable option.



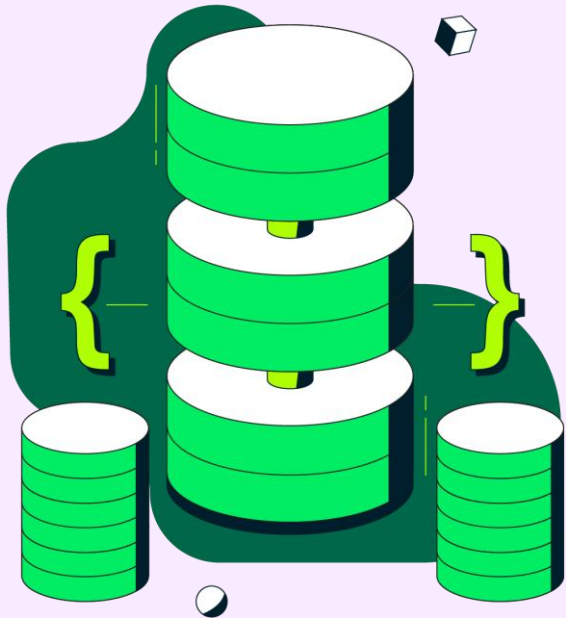
Recommendation 3



Arrays should not grow without bound. If there are more than a couple of hundred documents on the "many" side, don't embed them; if there are more than a few thousand documents on the "many" side, don't use an array of ObjectID references. High-cardinality arrays are a compelling reason not to embed.



Recommendation 4



How you model your data depends entirely on your particular application's data access patterns. You want to structure your data to match the ways that your application queries and updates it.



Design patterns

Brief introduction

What are Patterns?



Building Blocks

Identified by our Consulting Engineers helping customers for the last 12+ years.

Common Language

Data Architects and Engineers can easily reference the same things.

What Can Patterns Do?



Improve Performance

by using no more resources than you should

Simplify access to the data

by grouping and pre-arranging data in a simpler form

Patterns Overview

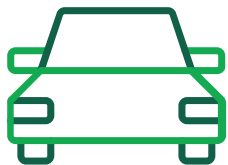
		Patterns											Related Examples	
		Approximation	Attribute	Bucket	Computed	Document Versioning	Extended Reference	Outlier	Preallocated	Polymorphic	Schema Versioning	Subset		Tree
Use Cases	Catalog	✓	✓		✓	✓	✓			✓	✓	✓	✓	Inventory Management
	Content Management		✓			✓				✓	✓	✓	✓	
	Internet Of Things	✓		✓	✓			✓	✓		✓			Log data, Time Series, Block Chain
	Mobile	✓			✓		✓	✓		✓	✓	✓		
	Personalization				✓	✓		✓			✓	✓		Point of sale, User Management
	Real-Time Analytics	✓		✓	✓		✓		✓		✓			Data Warehouse
	Single View		✓		✓	✓				✓	✓			



Polymorphic Pattern



Polymorphic pattern



```
{  
  "vehicle_type": "car",  
  "owner": "Roland",  
  "taxes": "200",  
  "wheels": 4  
}
```



```
{  
  "vehicle_type": "truck",  
  "owner": "Daniel",  
  "taxes": "800",  
  "wheels": 10,  
  "axles": 3,  
}
```



Pattern in practice: Single view use

System A

```
{  
  id: "203-102-1222",  
  name: "Adams",  
  first: "Samuel",  
  address: "100 Forest",  
  city: "Palo Alto",  
  state: "California",  
}
```

System B

```
{  
  ss: "203-102-1222",  
  last_name: "Adams",  
  first_name: "Samuel",  
  address: "100 Forest",  
  city: "Palo Alto",  
  state: "CA",  
}
```

System C

```
{  
  pkey: "123456",  
  soc_sec: "203-102-1222",  
  name: "Samuel Adams",  
  street: "222 University",  
  city: "Palo Alto",  
  state: "CA",  
}
```

Single View

```
{  
  _id: "203-102-1222",  
  insurance_types: ["life", "home",  
    "car"],  
  last_name: "Adams",  
  first_name: "Samuel",  
  addresses: [  
    { address: "100 Forest",  
      city: "Palo Alto", state:  
        "California" },  
    { address: "100 Forest",  
      city: "Palo Alto", state: "CA" },  
    { street: "222 University",  
      city: "Palo Alto", state: "CA" }  
  ]  
}
```



Polymorphism in the schema versioning pattern

```
{  
  id: "203-102-1222",  
  name: "Adams",  
  first: "Samuel",  
  address: "100 Forest",  
  city: "Palo Alto",  
  state: "California",  
  telephone: 400-900-4000,  
  cellphone: 600-900-0003  
}
```



```
{  
  id: ObjectID,  
  schema_verison: <int>,  
  name: <string>,  
  first: <string>,  
  address: <string>,  
  city: <string>,  
  state: <string>,  
  contacts: [  
    method: <string>,  
    value: <sting>  
  ]  
}
```



```
{  
  id: "203-102-1222",  
  schema_version: 1,  
  name: "Adams",  
  first: "Samuel",  
  address: "100 Forest",  
  city: "Palo Alto",  
  state: "California",  
  contacts: [  
    { method: telephone,  
      value: 400-900-4000 },  
    { method: cellphone,  
      value: 600-900-0003 },  
    { method: email,  
      value: sam.adams@mongodb.com }  
  ]  
}
```



Polymorphism pattern

Problem

Objects more similar than different
Want to keep objects in same collection

Solution

Field tracks the type of document or sub-document
Application has different code paths per document type, or has subclasses

Use Case Examples

Single View
Product Catalog
Content Management

Benefits and Trade-offs

Easier to implement
Allow to query across a single collection



Schema Versioning Pattern



Updating a Relational Database Schema

New schema is generated

Migration files, similar to git commits

Application is stopped and restarted with new schema

Hard revert to the old schema, if a problem arises

Document database — add a new field, no requirement for stopping, starting or migration files.



Schema versioning pattern

```
{  
  id: "203-102-1222",  
  name: "Adams",  
  first: "Samuel",  
  address: "100 Forest",  
  city: "Palo Alto",  
  state: "California",  
  telephone: 400-900-4000,  
  cellphone: 600-900-0003  
}
```



```
{  
  id: ObjectID,  
  schema_verison: <int>,  
  name: <string>,  
  first: <string>,  
  address: <string>,  
  city: <string>,  
  state: <string>,  
  contacts: [  
    method: <string>,  
    value: <sting>  
  ]  
}
```



```
{  
  id: "203-102-1222",  
  schema_version: 1,  
  name: "Adams",  
  first: "Samuel",  
  address: "100 Forest",  
  city: "Palo Alto",  
  state: "California",  
  contacts: [  
    { method: telephone,  
      value: 400-900-4000 },  
    { method: cellphone,  
      value: 600-900-0003 },  
    { method: email,  
      value: sam.adams@mongodb.com }  
  ]  
}
```



Schema versioning pattern

Problem

Avoid downtime doing schema upgrades
Upgrading all documents can take hours, days or even weeks when dealing with big data

Solution

Each document gets a "schema_version" field
Application can handle all versions
Choose your strategy to migrate the documents

Use Case Examples

Every application that use a database, deployed in production and heavily used.
System with a lot of legacy data

Benefits and Trade-offs

No downtime needed
Feel in control of the migration
Less future technical debt
May need 2 indexes for same field while in migration period



Attribute Pattern



Situations where this pattern is useful

a subset of fields share common characteristics and we want to sort or query on that subset of fields, or

a small subset of documents contain the fields we want to sort on, or

both of the above.



Pattern in practice: movie data

```
{
  title: "Star Wars",
  director: "George Lucas",
  ...
  release_US: ISODate ("1977-
05-20T01:00:00+01:00"),
  release_France: ISODate
("1977-10-19T01:00:00+01:00"),
  ...
}
```

```
{
  title: "Star Wars",
  director: "George Lucas",
  ...
  releases:[
    { location: "US",
      Date: ISODate ("1977-05-
20T01:00:00+01:00")},
    { location: "France",
      Date: ISODate ("1977-05-
20T01:00:00+01:00")},
    ...
  ]
}
```



Pattern in practice: Water bottles

```
{  
  "specs": [  
    { k: "volume", v: "500", u: "ml" },  
    { k: "volume", v: "12", u: "ounces" }  
  ]  
}
```

```
{"specs.k": 1, "specs.v": 1, "specs.u": 1}
```



Attribute pattern

Problem

Lots of similar fields

Want to search across many fields at once

Fields present in only a small subset of documents

Solution

Break the field/value into a sub-document

```
{ "color": "blue", "size": "large" }
```

```
{ [ { "k": "color", "v": "blue" },  
  { "k": "size", "v": "large" } ] }
```

Use Case Examples

Characteristics of a product

Set of fields all having same value type

List of dates

Benefits and Trade-offs

Easier to index

Allow for non-deterministic field names

Ability to qualify the relationship of the original field and value



Subset Pattern



Subset pattern: When and why it is needed?

Used when a large portion of data inside a document that is rarely needed.

Examples of when data is not part of the majority of the queries include product reviews, article comments, or cast in a movie.

Solution: split the frequently data and the rarely accessed data.

The Subset pattern is the solution to refactoring schemas with this characteristic.



Subset pattern

```
{  
  "name": "MongoDB- The Definitive Guide",  
  "edition": "3",  
  "description": "Learn all about MongoDB",  
  "price": { "value":NumberDecimal(29.99), "currency": "USD" },  
  "reviews": [ { rid: 767, username: "eliot_h", review: "Great introduction to  
                  MongoDB", date: ISODate("2020-09-09")},  
                { rid: 766, username: "dwight_m", review: "Fantastic overview of  
                  MongoDB", date: ISODate("2020-09-08")}, ...  
                { rid: 1, username: "kevin_r", review: "Nice introduction to  
                  MongoDB", date: ISODate("2020-01-06")}  
  ]  
}
```



Subset pattern: split the data

```
{ review_id: 740, product_id:
  ObjectId("5f2aefa8fde88235b959f0b1e"),
  review_author: "ken_a", review_date: ISODate("2020-08-08"),
  review_text: "Nice book, great topics!"},
```

```
{ review_id: 739, product_id:
  ObjectId("5f2aefa8fde88235b959f0b1e"),
  review_author: "matt_j", review_date: ISODate("2020-08-06"),
  review_text: "Fantastic book, learnt lots."},
```

```
{ review_id: 738, product_id:
  ObjectId("5f2aefa8fde88235b959f0b1e"),
  review_author: "sonalim", review_date: ISODate("2020-07-06"),
  review_text: "Comprehensive MongoDB coverage"}
```



Subset pattern

Problem

Working set is too big
Lot of pages are evicted from memory
A large part of documents is rarely needed

Solution

Split the collection in 2 collections
Most used part of documents
Less used part of documents
Duplicate part of a 1-N or N-N relationship that is often used in the most used side

Use Case Examples

List of reviews for a product
List of comments on an article
List of cast in a movie

Benefits and Trade-offs

Smaller working set, used documents are smaller
Shorter disk access for bringing in additional documents from the most used collection
Can add more round trips to the server
A little more space used on disk



Thank you for
your time.